

Министерство образования и науки Российской Федерации
Федеральное Государственное Автономное Образовательное
Учреждение Высшего Образования "Национальный
Исследовательский Университет ИТМО"

Факультет программной инженерии и компьютерной техники
Дисциплина "Системы ввода-вывода"

Лабораторная работа №4

Изучение работы контроллера
ввода/вывода

Выполнили
Волокитин Александр Сергеевич
Копытов Никита Эдуардович
Лаврентьев Лев Денисович
Группа Р3319

Проверил
Быковский Сергей Вячеславович

г. Санкт-Петербург, 2026 г

1. Введение

Цель работы: познакомиться с принципами работы контроллеров ввода/вывода на примере контроллера UART микроконтроллера ATmega328

Задачи:

1. Реализовать драйвер UART на уровне регистров аппаратного контроллера ATmega328 (вариант 2: датчик BMP280 по SPI, 38400 бод, odd parity, 2 стоп-бита)
2. Реализовать приём пакетов с проверкой целостности (CRC8) и отправкой эхо-ответа
3. Реализовать периодическую передачу данных датчика BMP280 (температура и давление) раз в секунду
4. Написать клиентскую программу на Python для тестирования корректных и некорректных пакетов
5. Подключить микроконтроллер к ПК и протестировать работу
6. Снять осциллограмму передачи пакета по интерфейсу UART

2. Формат пакета

Пакет состоит из четырёх полей, передаваемых последовательно:

Поле	Размер	Описание
S	1 байт	Синхробайт 0x5A - маркер начала пакета
LEN	1 байт	Количество байт в поле DATA (0–255)
DATA	LEN байт	Полезная нагрузка
CRC8	1 байт	Контрольная сумма (полином 0x07, init 0x00), считается по полям LEN + DATA

Пакет с данными датчика содержит 8 байт: float температура (°C) + float давление (Па), little-endian

3. Реализация

Полный исходный код: `uart/uart.ino` (микроконтроллер), `client/client.py` (клиент на ПК)

3.1. Программа микроконтроллера (`uart/uart.ino`)

Инициализация UART. Скорость задаётся через делитель UBRR, формат кадра - через биты регистров UCSR0B и UCSR0C:

```
uint16_t baudRate = 38400;
uint16_t ubrr = 16000000 / 16 / baudRate - 1;
UBRR0H = (unsigned char)(ubrr >> 8);
UBRR0L = (unsigned char) ubrr; // стр. 162
```

```

SetBit(UCSR0B, TXEN0);
SetBit(UCSR0B, RXEN0);
SetBit(UCSR0B, RXCIE0);

SetBit(UCSR0C, UCSZ01); // 8 бит данных стр. 162
SetBit(UCSR0C, UCSZ00);
SetBit(UCSR0C, UPM01); // odd parity стр. 161
SetBit(UCSR0C, UPM00);
SetBit(UCSR0C, USBS0); // 2 стоп-бита стр. 161

bmp.begin(); // инициализация датчика

```

Отправка байта - ожидание флага UDRE0 (буфер свободен), затем запись в UDR0:

```

// ждём освобождения буфера передатчика (UDRE0 в UCSR0A стр. 159),
// затем пишем в регистр данных UDR0 (стр. 159) - это запускает передачу
void uartSendByte(uint8_t b) {
    while (!(UCSR0A & (1 << UDRE0)));
    UDR0 = b;
}

```

Приём - прерывание ISR(USART_RX_vect). Каждый принятый байт обрабатывается конечным автоматом с четырьмя состояниями (S_SYNC -> S_LEN -> S_DATA -> S_CRC). При совпадении CRC пакет помечается готовым к эхо-ответу; битый пакет отбрасывается, автомат возвращается в S_SYNC

Данные датчика - раз в секунду loop() вызывает sendSensorPacket(), которая читает температуру и давление с BMP280 по SPI и отправляет их как два float (8 байт, little-endian) в штатном формате пакета

3.2. Клиентская программа (client/client.py)

Открытие порта:

```

ser = serial.Serial(
    PORT, 38400,
    bytesize=serial.EIGHTBITS,
    parity=serial.PARITY_ODD,
    stopbits=serial.STOPBITS_TWO,
    timeout=2,
)

```

CRC8 (полином 0x07, init 0x00) - идентичен реализации на МК; считается по полям LEN + DATA:

```

def crc8(data):
    crc = 0x00
    for byte in data:
        crc ^= byte

```

```

        for _ in range(8):
            crc = ((crc << 1) ^ 0x07) & 0xFF if (crc & 0x80) else (crc
<< 1) & 0xFF
        return crc

```

Тесты - функция `run_tests()` последовательно отправляет три варианта и слушает эхо в течение 1.5 с; для корректного пакета ожидается эхо, для остальных - нет. Результат выводится как OK / FAIL. Данные датчика читаются в `main()` через `read_packet()`: 8 байт распаковываются как `struct.unpack("<ff", data) ->` температура и давление

4. Тестирование

Клиентская программа последовательно отправляет три тестовых случая:

```

data = b"TEST"
correct = build_packet(data)
tests = [
    ("Корректный пакет", correct, data, True),
    ("Нет синхробайта", correct[1:], data, False),
    ("Недостаточно данных", bytes([SYNC, 8]) + b"AB", b"AB", False),
]

```

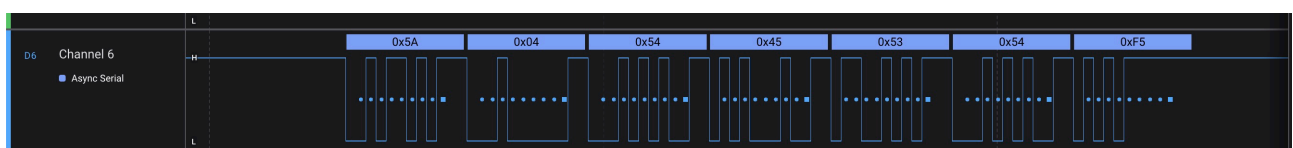
Тест	Описание	Результат
Корректный пакет	Пакет без ошибок, ожидается эхо	OK
Нет синхробайта	Первый байт 0x5A пропущен	OK
Недостаточно данных	LEN=8, передано только 2 байта	OK

Микроконтроллер принимает только первый тест и игнорирует остальные. После тестов в непрерывном режиме читаются данные датчика BMP280 и выводятся температура и давление

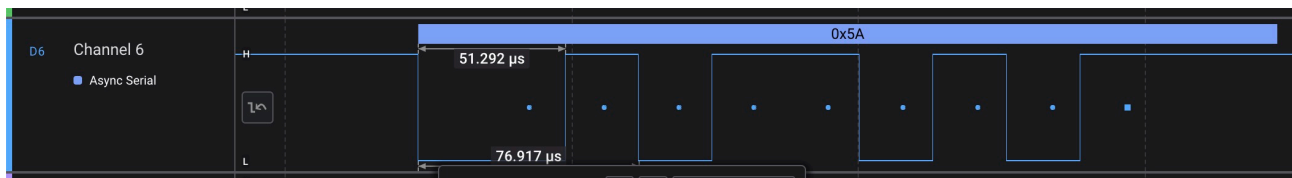
5. Осциллограмма

Захват выполнен в Logic 2. Анализатор Async Serial настроен параметры варианта: 38400 бод, 8 бит, odd parity, 2 стоп-бита. Показан пакет, отправленный от ПК к МК

Пакет целиком - декодированные байты на временной оси:



Увеличение на один кадр (синхробайт 0x5A):



Временные параметры при 38400 бод, формат 8-О-2 (1 старт + 8 данных + 1 чётность + 2 стоп = 12 бит на кадр):

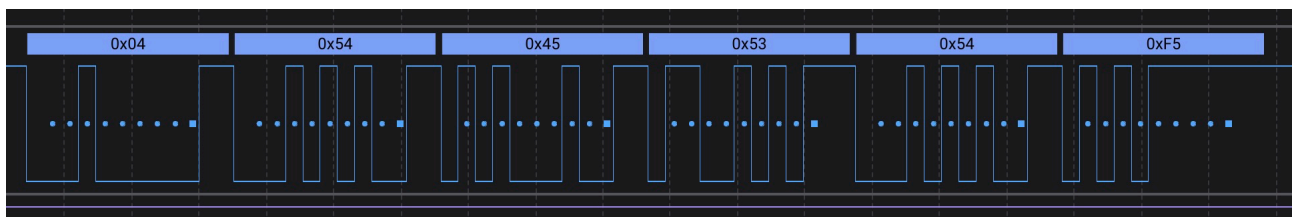
Параметр	Значение
Длительность одного бита	$1/38400 \approx 26.04$ мкс
Длительность кадра (12 бит)	$12 \times 26.04 \approx 312.5$ мкс
Пакет 7 байт (эхо «TEST»)	$7 \times 312.5 \approx 2.19$ мс
Пакет датчика 11 байт	$11 \times 312.5 \approx 3.44$ мс

Проверка контрольной суммы. Байты пакета считаны с осциллограммы: 5A 04 54 45 53 54 F5 (эхо строки «TEST»). Результат `crc_check.py`:

```
$ python3 client/crc_check.py 5A 04 54 45 53 54 F5
Синхробайт : 0x5A
Длина      : 4
Данные     : 54 45 53 54
CRC принят : 0xF5
CRC расчёт : 0xF5
=> CRC сошёлся, пакет целый
```

5.1. Некорректный пакет - отсутствие синхробайта

Передаётся последовательность 04 54 45 53 54 F5 без ведущего 0x5A



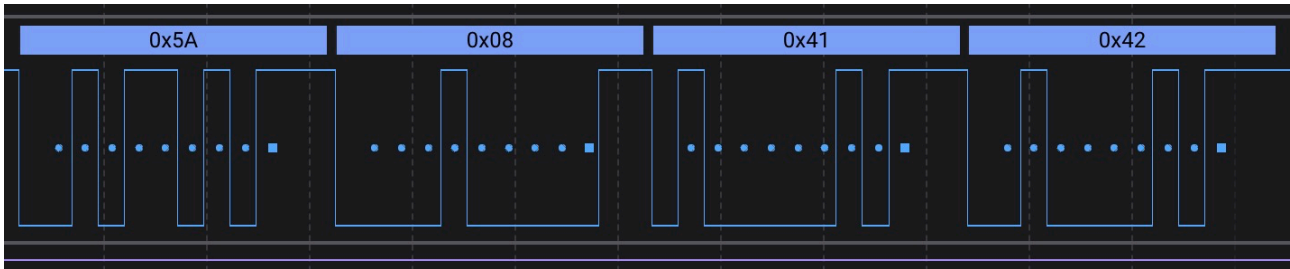
Контроллер находится в состоянии `S_SYNC` и ожидает байт `0x5A`. Первый байт `0x04` не совпадает с синхробайтом - все последующие байты также игнорируются, автомат остаётся в `S_SYNC`, эхо не отправляется

Проверка `crc_check.py` выявляет отсутствие синхробайта:

```
$ python3 client/crc_check.py 04 54 45 53 54 F5
Синхробайт : 0x04  <- ожидался 0x5A!
Длина      : 84  <- данных по факту 3!
Данные     : 45 53 54
CRC принят : 0xF5
CRC расчёт : 0x7A
=> CRC НЕ сошёлся, пакет битый
```

5.2. Некорректный пакет - неправильная длина

Передаётся 5A 08 41 42: синхробайт корректный, LEN = 8, но фактически следует только 2 байта данных (0x41, 0x42)



Контроллер принимает синхробайт, переходит в S_LEN, читает длину 8, затем входит в S_DATA и ожидает 8 байт. Получив только 2 (0x41 = „А“, 0x42 = „В“), автомат застревает в S_DATA - байты CRC так и не поступают, пакет не завершается, эхо не отправляется

Проверка `crc_check.py`:

```
$ python3 client/crc_check.py 5A 08 41 42
Синхробайт : 0x5A
Длина      : 8  <- данных по факту 1!
Данные     : 41
CRC принят : 0x42
CRC расчёт : 0x68
=> CRC НЕ сошёлся, пакет битый
```